TECHNICAL RESPONSE TO OES-ACC-01

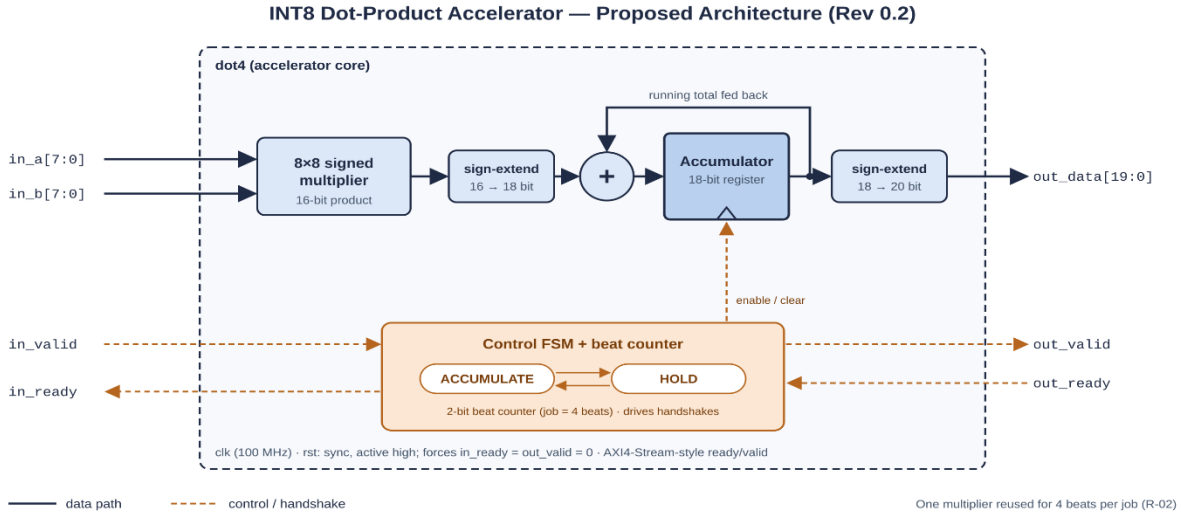
COMPACT INTEGER DOT-PRODUCT ACCELERATOR

The design takes four pairs of signed 8-bit numbers, multiplies each pair, and adds the four results: R = a0·b0 + a1·b1 + a2·b2 + a3·b3. The output is an exact 20-bit signed number, with no rounding or saturation. The design is written in SystemVerilog and runs on the Nexys A7-100T board at 100 MHz. The main goal is a correct, well-verified result; speed is less important. This is a proof of concept, not a final product.

Interfaces & architecture:

AXI4-Stream-style ready/valid interface (not the full standard AXI4-Stream signal set)

Port	Direction	Width	Description
clk	in	1	100 MHz clock
rst	in	1	Synchronous reset, active high
in_valid	in	1	Input pair is valid
in_ready	out	1	Design can accept a pair
in_a	in	8	Operand a, signed
in_b	in	8	Operand b, signed
out_valid	out	1	Result is valid
out_ready	in	1	Receiver can take the result
out_data	out	20	Result R, signed



Assumptions and Design Decisions:

- **Framing:** 4 beats per job, one (a, b) pair per beat; one shared multiplier (R-02).
- **Job end: each job is exactly 4 accepted beats, counted internally (2-bit counter).** ***Assumption:*** **the source always sends pairs in complete groups of four.**

- **Handshake:** AXI4-Stream-style ready/valid (transfer rules per Arm IHI 0051B, §2.2); no AXI IP. Transfer when valid = ready = 1. Inputs held stable by the source until accepted (assumption); out_valid/out_data held stable until out_ready (R-03)
- **Architecture:** 8×8 signed multiplier, 18-bit accumulator, 2-state FSM (ACCUMULATE, HOLD).
- **Widths:** range −65,024 … +65,536 → 18 bits, sign-extended to 20.
- **Latency:** Let *E4* be the clock edge where the 4th pair is transferred.
    - At *E4*, the accumulator stores the final sum and the FSM enters HOLD. **out_valid is high from the cycle after *E4*** (registered output).
    - At the edge where out_valid = out_ready = 1, the result is transferred, and the accumulator and counter are cleared.
    - **The next job cannot start in that same cycle.** in_ready rises in the cycle after the output transfer, so there's no combinational path from out_ready to in_ready. Minimum job period: 5 cycles.
- **Reset:** synchronous, active high; discards any job in progress or held result. Then out_valid = 0, in_ready = 1.

Verification Approach

- **Simulation:** self-checking testbench with an independent reference model (plain integer math). Directed tests: edge values (−128, +127, 0), the maximum case (all −128), reset mid-job, output stalls, back-to-back jobs. Random tests: random operands, input gaps and output stalls.
- **Formal (SVA):** checker modules attached via bind, separate from the RTL.
    - **Assumptions:** the input side follows ready/valid rules; pairs arrive in complete groups of four.
    - **Assertions:** a complete property set covering every output in every state (reset, handshake, state transitions, latency, result value), with a cover/witness for each. Completeness checked with the tool's coverage/completeness analysis where available. Arithmetic: result checked against a reference expression (reduced width; full width is a stretch goal).

#	Mandatory property	Type
P1	After reset: out_valid = 0, in_ready = 1	assert
P2	out_valid and out_data stable while out_ready = 0	assert
P3	out_valid only after 4 accepted beats	assert
P4	in_ready = 0 while a result is pending	assert
P5	Result = $\Sigma a_i \cdot b_i$ at reduced width (4-bit operands)	assert

    - **Non-vacuity:** a cover/witness for every assertion.
- **Tools:** OneSpin and/or VC Formal; SymbiYosys for open reproducibility.

- **Reporting:** all assumptions and proof bounds stated; bounded results are not claimed as full proofs.

Board Demo Plan

- **Built-in self-test:** an on-chip ROM holds test jobs and their expected results, both generated offline by the reference model. A test controller feeds each job to the accelerator and compares the result.
- **Controls:** BTNC starts the test; CPU_RESETN resets (synchronized in the board top).
- **Result:** RGB LED **green = PASS**, **red = FAIL**. The 7-segment display shows the number of jobs passed.
- **Fault injection:** switch SW0 corrupts one expected value, so the test must show FAIL. This proves the test can detect errors.
- **Backpressure:** switch SW1 adds random out_ready stalls, exercising R-03 on hardware.
- **No extra hardware:** only onboard buttons, switches, LEDs and the display are used (R-07).